SYNC

Formal Model for Online Scheduling
in Resource-Constrained Multi-Agent Workflows

Proposed Solution: An Online Primal–Dual Framework
with Dynamic Multidimensional Knapsack Constraints

Abstract

SYNC is a framework-independent runtime optimizer for multi-agent workflows. It selects task assignments and execution schedules under task dependencies, worker compatibility, resource capacities, and operational policies.

The problem is modeled as online resource-constrained scheduling over a dynamically revealed workflow. Scheduling proposals are separated from authoritative admission so that optimization remains independent of execution correctness.

This document extends the scheduling formulation with a concrete primal–dual policy: a local packing relaxation, scarcity-price updates, feasible integer recovery, and rolling-horizon evaluation. The coffee-shop specialization includes cart-level outcomes, repeated item instances, stateful reusable resources, elastic staffing, and usage-based Anthropic cost accounting. The algorithm is a proposed engineering specification; no unreported benchmark results or global approximation guarantees are assumed.

Contents

Part I: Problem Formulation **4**

1 Problem Statement **4**

2 System Model **4**

2.1 Tasks 4

2.2 Workers and Compatibility 5

3 Workflow Eligibility **5**

4 Finite-Batch Scheduling Formulation **5**

4.1 Decision Variables 5

4.2 Assignment and Compatibility 6

4.3 Release Times and Dependencies 6

4.4 Worker Capacity 6

4.5 Shared-Resource Capacity 6

4.6 Rate Limits and Budgets 6

5 Optimization Objectives **7**
5.1 Finite-Batch Objective 7
5.2 Fixed-Horizon Completion Objective 7

6 Online Scheduling Policy **7**
6.1 Rolling-Horizon Approximation 8

7 Practical Assignment Policy **8**

8 Admission and Synchronization **9**
8.1 Task Ownership 9
8.2 Resource Safety 9
8.3 Fencing and Result Acceptance 9
8.4 Idempotent Events 10
8.5 Conditional Progress 10

9 Evaluation Methodology **10**

Part II: Proposed Solution **11**

10 Design Rationale and Scope **11**

11 Coffee-Shop Specialization **11**
11.1 Carts, Lines, and Unique Item Instances 11
11.2 Task Decomposition and Dependency Graph 12
11.3 Compatible Batching 13
11.4 Roles, Workers, and Station Slots 13

12 Resource Lifecycles and Dynamic Capacity **14**
12.1 Ownership and Clean/Dirty State 14
12.2 Budget Reservations 14
12.3 Sliding-Window Admission 15

13 Operational Definition of the Target Objective **15**

14 Full-DAG Rolling-Horizon Value Estimation **16**

15 Dynamic Multidimensional Knapsack Subproblem **16**
15.1 Candidate Options and Capacity Dimensions 16
15.2 Normalization and Local Primal Problem 17

16 Primal–Dual Derivation and Scarcity Prices **17**
16.1 LP Relaxation and Dual 17
16.2 Reduced Dual and Concrete Update Rule 18

17 Feasible Integer Recovery and Dispatch **19**
17.1 Mandatory Maintenance and Checkpoints 19
17.2 Authoritative Admission Contract 20

18 Elastic Staffing and Bottleneck Response **20**
18.1 Worker Lifecycle 20
18.2 Role-Specific Examples 21

19 Event-Driven Algorithm and Computation Budget **21**

20 Anthropic Agent Execution and Tokenomics **22**
20.1 Per-Attempt Usage Ledger 22

21 Claude Comparison and Minimal Benchmark Protocol **23**
21.1 Controllers and Shared Conditions 23
21.2 Workloads and Ablations 24
21.3 Measurements and Reporting 24

22 Limitations, Assumptions, and Implementation Status **24**

23 Explanation **25**

A Minimum Reproducibility Record **26**

Part I: Problem Formulation

1 Problem Statement

Given a dynamically revealed workflow, heterogeneous workers, and constrained shared resources, find an online scheduling policy that improves completion value, latency, or execution cost while preserving dependency and resource-ownership invariants and accounting for coordination overhead.

We initially assume non-preemptive execution: once a task starts, it continues until completion or failure.

Tasks that release resources or support safe interruption between phases can be represented as multiple stages.

2 System Model

At decision time t, define the system as

$$\mathcal{S}_t = (G_t, \mathcal{A}, \mathcal{R}, X_t, \Theta_t)\,. \tag{1}$$

Symbol	Meaning
$G_t = (V_t, E_t)$	Known task instances and dependency edges
$\mathcal{A}$	Set of worker agents
$\mathcal{R}$	Set of shared resources
X_t	Current task, worker, and resource states
Θ_t	Estimates of duration, cost, and failure behavior

An edge $(i, j) \in E_t$ means that task j requires successful completion of task i.

The graph contains task instances rather than reusable workflow nodes. Repeating a workflow step creates a new task instance. Conditional branches expose tasks after their guards resolve.

2.1 Tasks

For each task $i \in V_t$, define

$$T_i = (r_i, d_i, w_i, v_i, K_i, \mathbf{q}_i, \mathcal{P}_i)\,, \tag{2}$$

where

- r_i: release time;
- d_i: optional deadline;
- $w_i \geq 0$: priority or latency weight;
- $v_i \geq 0$: value of successful completion;
- K_i: required capabilities;
- $q_{ir} \geq 0$: units of resource r required;
- $\mathcal{P}_i$: execution, permission, and retry policies.

Let $\mathcal{D} \subseteq V_t$ denote tasks with deadlines.

2.2 Workers and Compatibility

Each worker $a \in \mathcal{A}$ has capabilities K_a and concurrency limit m_a.

Define

$$\chi_{ia} = \begin{cases} 1, & \text{worker } a \text{ has the capabilities, context,} \\ & \text{and access required to execute task } i, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Let p_{ia} and c_{ia} denote execution duration and cost. Before execution, SYNC uses estimates $\widehat{p}_{ia}$ and $\widehat{c}_{ia}$.

3 Workflow Eligibility

Let

$$\mathrm{Pred}(i) = \{j : (j,i) \in E_t\}. \tag{4}$$

A task is ready when

$$\begin{aligned} \mathrm{Ready}(i,t) \iff\ & r_i \le t \\ & \wedge\ \mathrm{status}_i(t) = \mathrm{Pending} \\ & \wedge \bigwedge_{j \in \mathrm{Pred}(i)} [\mathrm{status}_j(t) = \mathrm{Succeeded}] \\ & \wedge\ \mathrm{WorkflowGuard}_i(X_t). \end{aligned} \tag{5}$$

The ready frontier is

$$\mathcal{F}_t = \{i \in V_t : \mathrm{Ready}(i,t)\}. \tag{6}$$

The agent framework supplies workflow guards and task semantics. SYNC selects assignments from the ready frontier while respecting these restrictions.

4 Finite-Batch Scheduling Formulation

First consider a finite batch of mandatory tasks with known durations and no failures. This defines the underlying scheduling problem. Online execution is addressed subsequently.

4.1 Decision Variables

Let

$$x_{ia} \in \{0,1\} \tag{7}$$

indicate whether worker a executes task i.

Let s_i and f_i be task start and finish times:

$$f_i = s_i + \sum_{a \in \mathcal{A}} p_{ia} x_{ia}. \tag{8}$$

4.2 Assignment and Compatibility

Each task is assigned exactly once:

$$\sum_{a \in \mathcal{A}} x_{ia} = 1 \qquad \forall i. \tag{9}$$

Assignments must be compatible:

$$x_{ia} \leq \chi_{ia} \qquad \forall i, a. \tag{10}$$

4.3 Release Times and Dependencies

$$s_i \geq r_i \qquad \forall i. \tag{11}$$

$$s_j \geq f_i \qquad \forall (i, j) \in E_t. \tag{12}$$

These precedence constraints assume successful execution. At runtime, readiness depends on committed successful outcomes.

4.4 Worker Capacity

For every worker a and execution time u,

$$\sum_i x_{ia} \mathbf{1}\{s_i \leq u < f_i\} \leq m_a. \tag{13}$$

The initial benchmark uses $m_a = 1$.

4.5 Shared-Resource Capacity

Let $C_r(u)$ be resource r's capacity at time u.

Require

$$\sum_i q_{ir} \mathbf{1}\{s_i \leq u < f_i\} \leq C_r(u) \qquad \forall r, u. \tag{14}$$

Exclusive ownership is the special case $C_r(u) = 1$.

This formulation assumes resources are held for the full task duration. Resource-specific phases should be modeled as stages.

4.6 Rate Limits and Budgets

For a resource allowing at most L_r requests within any window of length W_r, require

$$N_r([u, u + W_r)) \leq L_r \qquad \forall u, \tag{15}$$

where N_r counts admitted requests.

For execution budget B, require

$$\sum_{i,a} c_{ia} x_{ia} \leq B. \tag{16}$$

Uncertain costs require budget reservations and reconciliation against actual spending.

5 Optimization Objectives

5.1 Finite-Batch Objective

Define flow time

$$F_i = f_i - r_i \tag{17}$$

and tardiness

$$L_i = \max\{0, f_i - d_i\} \qquad i \in \mathcal{D}. \tag{18}$$

A possible objective is

$$\boxed{\min_{\pi \in \Pi_{\text{feasible}}} \mathbb{E}\Bigg[\alpha \sum_i w_i F_i + \beta \sum_{i \in \mathcal{D}} w_i L_i + \gamma C_{\text{exec}}(\pi) + \delta C_{\text{coord}}(\pi) + \eta C_{\text{change}}(\pi)\Bigg].} \tag{19}$$

Here

- C_{exec}: model, tool, and compute costs;
- C_{coord}: optimization and coordination overhead;
- C_{change}: supported reassignment or cancellation costs.

All weights are nonnegative and must account for the units and scales of their corresponding terms.

5.2 Fixed-Horizon Completion Objective

For benchmark horizon T, let $\mathcal{O}$ contain the terminal outcomes that earn value, such as completed orders.

An alternative objective is

$$\boxed{\max_{\pi \in \Pi_{\text{feasible}}} \mathbb{E}\Bigg[\sum_{i \in \mathcal{O}} v_i \mathbf{1}\{f_i \le T \ \wedge \ \text{Success}(i)\} - \beta P_{\text{deadline}}(\pi) - \gamma C_{\text{exec}}(\pi) - \delta C_{\text{coord}}(\pi)\Bigg].} \tag{20}$$

Intermediate tasks need not earn independent rewards. For the coffee-shop specialization, one customer order is a cart. Value is attached once to successful delivery of the complete cart. Menu-item instances and support tasks do not independently earn the cart reward. Equation (20) is the primary target in Part II.

6 Online Scheduling Policy

Let $\mathcal{H}_t$ contain all information observed through decision epoch t.

A nonanticipative policy μ chooses

$$U_t = \mu(\mathcal{H}_t), \tag{21}$$

where U_t is the proposed set of assignments.

The system evolves as

$$X_{t+1} = F(X_t, U_t, \xi_t), \tag{22}$$

where ξ_t represents uncertain arrivals, outcomes, and resource changes.

The online control problem is

$$\mu^* = \arg\min_{\mu} \mathbb{E}_{\mu} \left[\sum_{t=0}^{T-1} \ell(X_t, U_t) + \Phi(X_T) \right], \tag{23}$$

subject to feasibility at every decision.

The terminal cost Φ accounts for unfinished work. The stage cost ℓ must be defined so that its accumulated value matches the selected operating objective.

6.1 Rolling-Horizon Approximation

At each decision epoch, solve

$$\widehat{\pi}_t = \arg \min_{\pi \in \Pi_{\text{feasible}}(X_t)} \widehat{J}_H(\pi \mid \mathcal{H}_t), \tag{24}$$

then execute only

$$U_t = \text{FirstActions}(\widehat{\pi}_t). \tag{25}$$

Observe the resulting events and replan.

Future arrivals used during planning must come from forecasts; the scheduler must not access hidden future benchmark events.

7 Practical Assignment Policy

The following weighted policy is a useful baseline. Its fixed weights $\theta_1, \ldots, \theta_6$ are distinct from the resource-price variables λ_k introduced later. A lightweight policy scores feasible task–worker pairs:

$$\begin{aligned} P(i, a, t) = {} & \theta_1 \, \text{Urgency}_i(t) + \theta_2 \, \text{Criticality}_i(t) \\ & + \theta_3 \, \text{Age}_i(t) - \theta_4 \widehat{p}_{ia} \\ & - \theta_5 \widehat{W}_{ia} - \theta_6 \widehat{c}_{ia}. \end{aligned} \tag{26}$$

Here $\widehat{W}_{ia}$ estimates resource waiting.

Estimated slack can inform urgency:

$$\text{Slack}_i(t) = d_i - t - \widehat{\text{Remaining}}\text{Path}_i(t). \tag{27}$$

Smaller slack implies greater urgency. Tasks without deadlines omit this term.

Let $y_{ia} \in \{0, 1\}$ indicate an immediate assignment. Choose

$$\max_{\mathbf{y}} \sum_{i,a} P(i, a, t) y_{ia} \tag{28}$$

subject to

$$\sum_{a} y_{ia} \le 1 \qquad \forall i, \tag{29}$$

$$\sum_{i} y_{ia} \le \text{FreeSlots}_a(t) \qquad \forall a, \tag{30}$$

$$\sum_{i,a} q_{ir} y_{ia} \le \text{FreeCapacity}_r(t) \qquad \forall r, \tag{31}$$

$$y_{ia} \le \mathbf{1}\{i \in \mathcal{F}_t\}\, \chi_{ia} \qquad \forall i, a. \tag{32}$$

Unselected tasks remain queued. An aging term can reduce starvation, although progress guarantees require additional assumptions about arrivals and capacity.

8 Admission and Synchronization

Optimization uses a versioned snapshot:

$$Q = \text{Plan}(X^{\nu}). \tag{33}$$

Admission revalidates the proposal against current authoritative state:

$$U = \text{Admit}(Q, X_{\text{current}}). \tag{34}$$

Admission atomically checks task ownership and resource availability, creates accepted claims, and issues dispatch tickets. Only admitted assignments may execute.

8.1 Task Ownership

For every task i,

$$\boxed{|\text{ActiveClaims}(i,t)| \le 1.} \tag{35}$$

This bounds authoritative current claims. An expired worker may still be physically running and must be handled through fencing or reconciliation.

8.2 Resource Safety

For every resource r,

$$\boxed{\text{AdmittedUsage}_r(t) \le C_r(t).} \tag{36}$$

8.3 Fencing and Result Acceptance

Let g_{ik} be the monotonically increasing fencing token for attempt k of task i.

Require

$$\begin{aligned} \text{AcceptResult}(i,k) \implies\ & g_{ik} = \text{CurrentToken}(i) \\ & \wedge\ \text{ValidTransition}(i,k). \end{aligned} \tag{37}$$

Reject results from superseded attempts.

8.4 Idempotent Events

For event e,

$$\text{Apply}\left(e, \text{Apply}(e, X)\right) = \text{Apply}(e, X). \tag{38}$$

This prevents duplicate delivery from applying the same authoritative transition twice.

These properties do not guarantee exactly-once external effects. External operations require destination-side deduplication, fencing, or outcome reconciliation.

8.5 Conditional Progress

A progress property can be written in temporal logic as

$$\Box\left(\text{PersistentlyEligible}(i) \Rightarrow \Diamond\, \text{Started}(i)\right), \tag{39}$$

where $\Box$ means "always" and $\Diamond$ means "eventually."

This property requires sufficient eventual capacity, a live compatible worker, bounded failures, and fair scheduling.

9 Evaluation Methodology

For workload w, random seed s, and policy μ, measure

$$M_\mu(w, s). \tag{40}$$

Compare baseline μ_B and SYNC μ_S while holding constant the workers, admission rules, resources, and exogenous arrivals.

For a higher-is-better metric with positive baseline mean,

$$\Delta_M = \frac{\overline{M}_{\mu_S} - \overline{M}_{\mu_B}}{\overline{M}_{\mu_B}} \times 100\%. \tag{41}$$

For a lower-is-better metric,

$$\Delta_M^{\text{reduction}} = \frac{\overline{M}_{\mu_B} - \overline{M}_{\mu_S}}{\overline{M}_{\mu_B}} \times 100\%. \tag{42}$$

Report paired-seed results and variability for:

- completed outcome value or throughput;
- completion latency;
- deadline misses;
- resource waiting and utilization;
- execution cost;
- scheduling and coordination overhead.

Report simulated execution time separately from wall-clock runtime. Improvement in throughput alone does not establish lower monetary cost.

Part II: Proposed Solution

10 Design Rationale and Scope

SYNC coordinates a team around completed outcomes. A locally productive agent can create a globally expensive queue. In the coffee shop, accepting more orders is useful only when preparation, delivery, and resource recycling can keep pace. The optimization target is Equation (20): completed value less deadline penalties, execution expense, and coordination overhead.

The proposed policy combines three ideas:

1. **Online control:** observe arrivals and committed outcomes, choose the next action, and replan without access to hidden future events.
2. **Dynamic multidimensional packing:** select assignments that simultaneously fit worker, station, equipment, rate, and budget limits.
3. **Primal–dual resource pricing:** maintain a feasible execution decision on the primal side and scarcity prices on the dual side.

Online packing provides a theoretical starting point for assigning scarce capacity as requests are revealed [1]. Models with departures also motivate treating reusable capacity as occupied for an interval, rather than permanently consumed [2]. These connections motivate SYNC; they do not transfer the cited papers' performance bounds to this complete system.

The entire problem is an online scheduling problem with precedence and stateful resources. A knapsack describes a local assignment subproblem. The proposed algorithm approximately improves Equation (20); it does not claim an exact solution to the stochastic online problem. Finiteness does not imply tractability. General scheduling and the local integer packing subproblem can be NP-hard.

11 Coffee-Shop Specialization

11.1 Carts, Lines, and Unique Item Instances

Let $\mathcal{C}_t$ be the carts observed by time t. A cart c contains lines $\mathcal{L}_c$. Line ℓ records a recipe key, recipe version, customization options, and quantity $n_{c\ell} \in \mathbb{N}_{>0}$. The physical item-instance set is

$$\mathcal{I}_c = \{(c, \ell, k) : \ell \in \mathcal{L}_c,\ 1 \le k \le n_{c\ell}\}. \tag{43}$$

Thus five coffees create five distinct instances. A unique request key deduplicates retries of cart creation; it does not collapse a requested quantity. A stable logical task key identifies the cart, line, item unit, and stage; a separate attempt identifier and fencing generation identify each retry. Retrying does not create a second logical task or reward.

For this demo, the terminal outcome is successful delivery of the complete cart. Write r_c, d_c, w_c, V_c for arrival, deadline, priority, and cart value. The set $\mathcal{O}$ in Equation (20) consists of cart-delivery outcomes. If prices are item-based, V_c may equal the sum of item values. An item, batch, payment, or cleaning stage never separately receives V_c.

11.2 Task Decomposition and Dependency Graph

The recipe supplies the exact preparation DAG; a latte may require cup preparation, espresso extraction, milk steaming, and finishing, whereas a black coffee omits milk-dependent stages. The common cart structure is:

1. **Validate cart:** verify recipes, options, quantity, and identifiers.
2. **Confirm payment:** create one cart-level payment outcome.
3. **Prepare item instances:** instantiate each recipe DAG, gated by successful validation and payment. Acquire clean equipment as required.
4. **Finish and stage each item:** produce one identified output per requested instance and return it to its original cart.
5. **Assemble cart:** wait for every required item output and acquire the cart's tray or hand-off reservation.
6. **Deliver cart:** commit one delivery outcome and its timestamp.
7. **Customer release:** record the observed or modeled release of the tray, cups, or table; release is not assumed at delivery.
8. **Clean and restore:** complete the necessary cleaning stages before reusable objects become available for a new owner.

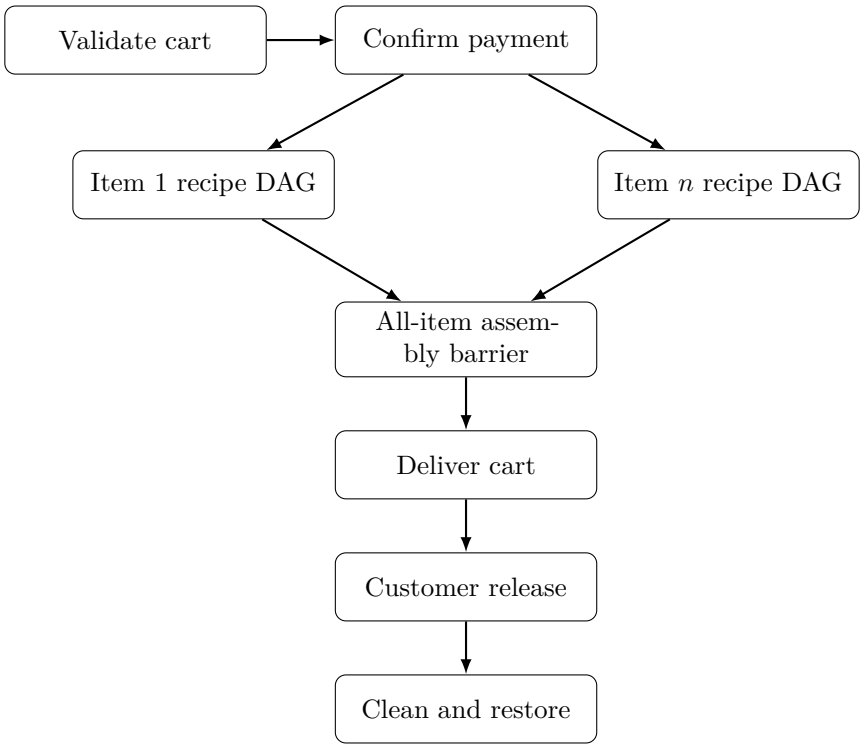


Figure 1: Cart-level structure. Each item branch expands into its recipe stages. The assembly barrier includes every item, including repeated units.

For instance $j \in \mathcal{I}_c$, let ItemReady(j) denote a committed successful final preparation stage. Then

$$\text{Ready}(\text{Assemble}(c), t) \implies \bigwedge_{j \in \mathcal{I}_c} \text{ItemReady}(j, t). \tag{44}$$

The cart retains its original arrival and deadline through decomposition, queueing, batching, and reassembly. Slowing the cashier cannot reset the SLA clock.

11.3 Compatible Batching

A batch b contains a declared membership set M_b of item-stage instances. Batch compatibility requires matching permitted recipe operations, options, equipment, quantity limits, and all member prerequisites. Every member retains its original (c, ℓ, k) identity.

On successful batch completion,

$$\forall j \in M_b : \quad |\text{CommittedOutputs}(b, j)| = 1. \tag{45}$$

Outputs are scattered to their original carts. Batch claims and individual claims for the same member stage are mutually exclusive. The basic packing derivation below uses one canonical task per option. An implementation may materialize a canonical batch task after atomically reserving its members. These reversible membership reservations are distinct from execution claims. A batch-formation lease or explicit rollback releases them if the batch is never admitted, without releasing any still-running external work. If batch alternatives are optimized simultaneously, overlapping membership requires additional conflict constraints; the simple per-task dual must then be extended accordingly.

11.4 Roles, Workers, and Station Slots

Let $\mathcal{K} = \{\text{cashier}, \text{barista}, \text{server}, \text{cleaner}\}$. The station capacities are

$$(K_{\text{cashier}}, K_{\text{barista}}, K_{\text{server}}, K_{\text{cleaner}}) = (2, 2, 2, 1). \tag{46}$$

There are seven potential logical workers, with the same home-role maxima. Initially one worker of each role is active, so four workers are active and three optional worker positions are off. Each worker runs at most one task at a time. A logical worker position is distinct from an API request.

Let $z_a(t)$ indicate that worker a is available for new assignments and $\kappa(i)$ be the station duty of task i. Immediate assignments satisfy

$$\sum_i y_{ia} \le \text{FreeSlots}_a(t) z_a(t), \tag{47}$$

$$\sum_{i,a:\kappa(i)=k} y_{ia} \le K_k - \text{OccupiedStationSlots}_k(t). \tag{48}$$

Helping another role requires $\chi_{ia} = 1$ and consumes the station capacity of the duty actually performed. A server helping the cashier does not create an extra cash-register slot or gain barista permissions.

12 Resource Lifecycles and Dynamic Capacity

Resource class	Examples	Capacity recovery
Renewable concurrency	Worker and station slots	Committed release
Reusable inventory	Trays, pitchers, cups	Release followed by required cleaning
Consumable inventory	Ingredients	Explicit replenishment only
Cumulative spending	API and execution budget	Does not return after completion
Sliding-window limits	API request allowance	Earlier requests leave the window

12.1 Ownership and Clean/Dirty State

For a reusable resource class r, partition its physical inventory into clean-free, held, dirty-free, and quarantined/uncertain objects:

$$N_r = N_r^{\text{clean}}(t) + N_r^{\text{held}}(t) + N_r^{\text{dirty}}(t) + N_r^{\text{uncertain}}(t). \tag{49}$$

An object is in exactly one state, and a held object has exactly one authoritative owner. Ownership may belong to a task, batch, or cart and may span several task stages. A cart's tray is acquired once; tasks using that existing claim do not acquire or charge another tray.

Cleaning consumes worker and equipment capacity now. It restores clean availability only after a successful committed state transition:

$$\text{DirtyFree} \longrightarrow \text{HeldForCleaning} \longrightarrow \text{CleanFree}. \tag{50}$$

It is not represented as negative immediate resource demand. A forecast release is not an authoritative release. Uncertain external operations retain their claims until fenced or reconciled safely.

12.2 Budget Reservations

Let S_t be reconciled spending, R_t outstanding reservations, and B the run's spending limit. Available uncommitted budget is

$$B_t^{\text{free}} = B - S_t - R_t. \tag{51}$$

Admission reserves $\bar{c}_o$ for a new operation and requires $\bar{c}_o \le B_t^{\text{free}}$. Reconciliation replaces its reservation with observed usage-based spending. Retries need their own reservations. Uncertain attempts retain a conservative reservation rather than becoming zero-cost attempts.

A hard budget guarantee requires a valid upper bound on every admitted charge, including bounded input, output, tools, and retries. If $\bar{c}_o$ is only a prediction, the policy is budget-aware but a hard provider-spend guarantee does not follow. Recovery and mandatory cleanup receive an explicit reserved allowance; an undersized allowance is reported as an infeasible run configuration.

If a task-level envelope is reserved, per-request reservations draw from that envelope rather than adding its full amount a second time. Planner requests also reserve budget before they are sent. Part I's execution-only budget can be extended to a shared run budget by including both execution and coordination charges in S_t and R_t.

12.3 Sliding-Window Admission

For a unit-request API limit, current immediate allowance is

$$L_r - N_r((t - W_r, t]). \tag{52}$$

An atomic batch of new requests must fit that allowance. In forecasting, request timestamps are retained so allowance recovers as old requests leave the window. Token-per-minute and request-per-minute constraints are separate dimensions when both apply. Every actual request, including subsequent agent turns, passes admission again.

13 Operational Definition of the Target Objective

Write $J_T(\pi)$ for the realized expression inside the expectation in Equation (20). The demo computes

$$\begin{aligned} J_T(\pi) = & \sum_{c \in \mathcal{C}_T} V_c \mathbf{1}\{f_c \le T \ \wedge \ \text{Delivered}(c)\} \\ & - \beta P_T - \gamma C_{\text{exec}} - \delta C_{\text{coord}}. \end{aligned} \tag{53}$$

Here $f_c = +\infty$ if delivery has not succeeded. One explicit deadline penalty that includes overdue unfinished carts is

$$P_T = \sum_{c : d_c \text{ defined}} w_c \left[(\min\{f_c, T\} - d_c)_+ + \rho \, \mathbf{1}\{d_c \le T < f_c\} \right], \qquad \rho \ge 0. \tag{54}$$

The first term counts lateness observed by T; the second optionally penalizes still-undelivered overdue carts. Lost completion value and this explicit penalty make unfinished work visible. Unfinished carts whose deadlines are after T lose the horizon completion reward but do not incur an overdue penalty. The benchmark fixes this definition before comparison.

Equation (20) does not directly minimize every cart's flow time. Mean and tail latency are companion measurements. If a waiting-time penalty is added to P_T, that is a declared objective variant, applied to every policy with the same weights.

All terms are converted into documented score units. In particular, simulated order value is not reported as real dollar revenue, and Anthropic dollars are not silently added to simulation ticks. Weights β, γ, δ provide the chosen conversions.

Use disjoint accounting categories:

$$\begin{aligned} C_{\text{exec}} &= C_{\text{role agents}} + C_{\text{execution tools}} + C_{\text{execution compute}}, \\ C_{\text{coord}} &= C_{\text{planner API}} + C_{\text{scheduler compute}} + C_{\text{coordination services}}. \end{aligned} \tag{55}$$

Each charge belongs to exactly one category. Planning wall time is also reported directly; it influences completion times when the benchmark execution model includes that delay. Charging compute and observing its latency are distinct declared effects, not duplicate ledger entries.

For the primary finite-horizon report, scored costs are all charges for physical requests and operations admitted by time T, reconciled even if their final usage arrives after T, plus compute/services accrued through T. Recovery first admitted after T is reported in a separate lifecycle-inclusive companion score. This convention fixes the cost boundary for both controllers; it does not make unfinished cleanup free.

14 Full-DAG Rolling-Horizon Value Estimation

At event t, forecast over H execution-time units, capped at T. The forecast includes all currently known DAG nodes, including blocked descendants, existing ownership intervals, expected release/cleaning, worker startup, estimated durations, and remaining budgets. Execution still starts only from $\mathcal{F}_t$.

Let $\widehat{Q}_t(S)$ be the estimated remaining contribution to the target when the immediate action set S is forced, followed by a fixed documented rollout policy. The baseline $\widehat{Q}_t(\varnothing)$ defers new launches until the next decision event; it need not idle for the entire horizon. For candidate option $o = (i, a)$, define

$$\widehat{\Delta J}_t(o \mid S) = \widehat{Q}_t(S \cup \{o\}) - \widehat{Q}_t(S). \tag{56}$$

Use the same sampled uncertainty scenarios in both counterfactuals. An alternative implementation may use a learned value estimator, but it must state its inputs, calibration, and uncertainty.

The estimator attaches value to completed terminal outcomes. Cleaning a pitcher can have positive marginal value because it enables several cart deliveries, even though the cleaning task itself earns no terminal reward. Predicted benefit is never inserted into the realized reward ledger.

If $t + H < T$, the truncated rollout may use a tail-value approximation for known unfinished work. That approximation estimates future target value; it is not an extra benchmark reward. A short horizon without a tail model can miss long prerequisite chains, so horizon length is an evaluated design choice. Mandatory safety/recovery work remains an operational obligation even when its estimated short-horizon benefit is small.

Unknown future arrivals are either excluded or sampled from a declared forecast fitted only to $\mathcal{H}_t$. Hidden benchmark traces are never available to either controller.

15 Dynamic Multidimensional Knapsack Subproblem

15.1 Candidate Options and Capacity Dimensions

The immediate compatible assignment options are

$$\mathcal{E}_t = \{o = (i, a) : i \in \mathcal{F}_t,\ \chi_{ia} = 1,\ a \text{ available for admission}\}. \tag{57}$$

Let $\mathcal{E}_t(i)$ contain the alternatives for task i. Each option starts a currently ready stage now. A forecast schedule for blocked work supplies utility estimates; it is not an immediate assignment.

Let $k \in \mathcal{D}_t^{\text{pack}}$ index packing dimensions. Dimensions include individual worker capacity, duty-station capacity, incremental equipment acquisition, remaining consumables, budget, and API allowance. A temporal forecast extension uses resource/time-bucket dimensions $k = (r, h)$ for $h = 0, \ldots, H_b - 1$.

Let $\widetilde{C}_{kt}$ be modeled capacity and $R_{kt}^{\text{committed}}$ existing commitments. Define residual capacity

$$B_{kt} = \widetilde{C}_{kt} - R_{kt}^{\text{committed}} \geq 0. \tag{58}$$

If an incident makes an existing commitment exceed capacity, ordinary new admission stops and an explicit recovery path is used. Capacity cannot be made nonnegative merely by forgetting ownership.

Equation (58) uses capacity measured before reservations. An already-free inventory field needs no second subtraction: for example, $B_{\text{clean},t} = N_r^{\text{clean}}(t)$, worker residual capacity is its total limit less current occupancy, and the budget residual is $B - S_t - R_t$. Subtracting held objects again from a clean-free count would understate availability.

Let $A_{ko}(t) \geq 0$ be additional demand from option o. For a renewable resource and time bucket, a conservative holding profile is

$$A_{(r,h),o}(t) = q_{or}^{\text{new}} \mathbf{1}\{\text{the predicted holding interval overlaps bucket } h\}. \tag{59}$$

Use the maximum occupancy within a bucket if a boundary cuts an interval. For a budget dimension, $A_{\text{budget},o} = \bar{c}_o$ once, not once per time bucket. Already-held cart resources are subtracted into commitments and do not appear again as new acquisition.

Future capacity or holding profiles are forecasts. Only the current-time constraints use authoritative current availability. At the next event, all constraints are rebuilt from observed state. Generic additive profiles cannot exactly represent every stateful interaction; the rollout handles replenishment and shared ownership explicitly.

15.2 Normalization and Local Primal Problem

Choose fixed positive physical scales s_k and objective scale $J_{\text{ref}} > 0$. Define

$$a_{ko} = \frac{A_{ko}}{s_k}, \qquad b_{kt} = \frac{B_{kt}}{s_k}, \qquad g_o = \frac{\widehat{\Delta J}_t(o \mid \varnothing)}{J_{\text{ref}}}. \tag{60}$$

Do not normalize by B_{kt}, which may be zero. Within a local optimization call, a, b, and g are frozen. The local integer packing problem is

$$\begin{aligned} \max_{\mathbf{y}} \quad & \sum_{o \in \mathcal{E}_t} g_o y_o \\ \text{subject to} \quad & \sum_{o \in \mathcal{E}_t(i)} y_o \leq 1 && \forall i \in \mathcal{F}_t, \\ & \sum_{o \in \mathcal{E}_t} a_{ko} y_o \leq b_{kt} && \forall k \in \mathcal{D}_t^{\text{pack}}, \\ & y_o \in \{0,1\} && \forall o \in \mathcal{E}_t. \end{aligned} \tag{61}$$

This is a multiple-choice multidimensional knapsack/packing problem: task–worker alternatives compete for several capacities at once. Unselected tasks remain pending.

The additive objective is a local surrogate. Two preparation tasks may contribute to the same cart, so their independent marginal values need not sum to the joint improvement in Equation (20). Feasible recovery therefore recomputes joint marginals when practical and evaluates complete candidate action sets through the rollout.

16 Primal–Dual Derivation and Scarcity Prices

16.1 LP Relaxation and Dual

Relax $y_o \in \{0,1\}$ to $y_o \geq 0$ in Equation (61). The per-task constraint already implies $y_o \leq 1$. Associate $u_i \geq 0$ with each task-exclusivity constraint and $\lambda_k \geq 0$ with each resource constraint.

The Lagrangian is

$$\begin{aligned}\mathcal{L}(\mathbf{y}, \mathbf{u}, \boldsymbol{\lambda}) = & \sum_i u_i + \sum_k b_{kt}\lambda_k \\ & + \sum_{o=(i,a)} y_o \left(g_o - u_i - \sum_k \lambda_k a_{ko} \right).\end{aligned} \tag{62}$$

Taking the supremum over nonnegative $\mathbf{y}$ gives the dual

$$\begin{aligned}\min_{\mathbf{u},\boldsymbol{\lambda}} \quad & \sum_i u_i + \sum_k b_{kt}\lambda_k \\ \text{subject to} \quad & u_i + \sum_k \lambda_k a_{ko} \geq g_o \quad \forall o \in \mathcal{E}_t(i), \\ & u_i \geq 0, \quad \lambda_k \geq 0.\end{aligned} \tag{63}$$

The resource price λ_k is expressed in normalized objective units per normalized capacity unit. It captures the opportunity cost of occupying that capacity. A high price guides selection; it does not grant permission to violate a hard constraint.

16.2 Reduced Dual and Concrete Update Rule

Define the priced profit

$$\sigma_o(\boldsymbol{\lambda}) = g_o - \sum_k \lambda_k a_{ko}. \tag{64}$$

Eliminating u_i yields the convex dual function

$$D_t(\boldsymbol{\lambda}) = \sum_k b_{kt}\lambda_k + \sum_{i \in \mathcal{F}_t} \max \left\{ 0, \max_{o \in \mathcal{E}_t(i)} \sigma_o(\boldsymbol{\lambda}) \right\}. \tag{65}$$

Empty alternative sets contribute zero and select no option. At inner iteration m, choose the best priced option independently for each task with $\mathcal{E}_t(i) \neq \varnothing$:

$$o_i^{(m)} \in \arg\max_{o \in \mathcal{E}_t(i)} \sigma_o(\boldsymbol{\lambda}^{(m)}). \tag{66}$$

Set $\widetilde{y}^{(m)}_{o_i^{(m)}} = 1$ if its priced profit is strictly positive, and select none for that task otherwise. Ties use a fixed stable identifier order. The oracle ignores cross-task resource constraints; its proposed demand may exceed capacity.

Projected subgradient descent on D_t is

$$\boxed{\lambda_k^{(m+1)} = \left[\lambda_k^{(m)} + \eta_m \left(\sum_o a_{ko} \widetilde{y}_o^{(m)} - b_{kt} \right) \right]_+, \qquad \eta_m > 0.} \tag{67}$$

For example, use $\eta_m = \eta_0/\sqrt{m+1}$ for a fixed maximum of M inner iterations. Here m indexes computation at one event; it is not execution time t.

Excess proposed demand raises the resource price; spare capacity lowers it, subject to nonnegativity. Using only admitted demand minus capacity would never raise prices while admission remains feasible, so it is not the update in Equation (67).

At a new event, recompute the snapshot, candidates, and residual capacities. Warm-start prices only for dimensions with unchanged meaning and scale. Shift time-bucket prices when the horizon advances; initialize new dimensions explicitly. Price caps or alternative utilization-target updates may be practical variants but are declared heuristic modifications.

Proposition 1 (Local weak-duality bound)**.** *For frozen local data, every feasible integer assignment set has objective at most $D_t(\boldsymbol{\lambda})$ for every $\boldsymbol{\lambda} \geq 0$.*

Proof. Choose $u_i = \max\{0, \max_{o \in \mathcal{E}_t(i)} \sigma_o\}$. Then $(\mathbf{u}, \boldsymbol{\lambda})$ is dual feasible. Weak duality bounds the relaxed primal and hence its feasible integer solutions. □

This is a bound on the frozen additive subproblem. It is not a bound on the unknown offline optimum of the complete workflow or on the realized stochastic objective. A finite iteration budget and integer recovery do not establish convergence or a competitive ratio for SYNC.

17 Feasible Integer Recovery and Dispatch

The oracle $\widetilde{\mathbf{y}}$ is never dispatched directly. The proposed fast implementation constructs a feasible set S as follows:

1. Start with $S = \varnothing$ and the residual capacities in Equation (58). Consider the best-price oracle candidates, a previous feasible plan, and the empty action as candidate sets.
2. Rank options by priced profit from Equation (64), with stable urgency/age tie breaks. Consider only currently compatible, ready options.
3. Tentatively add an option only if task exclusivity and every constraint of the frozen packing instance remain feasible, including modeled future buckets. Check batch-member conflicts and resource ownership explicitly. The local dual bound applies only to sets feasible for this same instance; current admission safety is a separate property.
4. When affordable, recompute $\widehat{\Delta J}_t(o \mid S)$ before adding the option. Reject an addition with negative estimated joint benefit. A nonnegative priced tie may be accepted; equality does not imply that work is useless.
5. Evaluate the recovered sets with the same full-DAG rollout and retain the feasible set with highest estimated target value, including waiting as an alternative. On equal predicted target values, prefer useful ready work using an oldest-eligible, work-conserving tie rule, so a zero marginal horizon score need not cause indefinite idling. A bounded joint search can replace greedy recovery for the seven-worker demo.
6. Submit the selected set, snapshot version, request key, and expected claims to atomic authoritative admission.

Recomputing marginal values makes recovery a heuristic beyond the frozen LP derivation; the local dual bound must still use the original frozen coefficients. An optional aging bonus may alter ranking to reduce starvation, but is identified as a fairness heuristic rather than a new terminal reward. No unconditional starvation guarantee is claimed.

17.1 Mandatory Maintenance and Checkpoints

Recovery jobs needed to restore a safe or reusable state are retained in the workflow even after the last delivery. Closing arrivals does not erase cleanup obligations. If all customer-facing gains are small, a separately declared recovery rule may dispatch required cleanup using its reserved budget. Such work still passes all hard admission checks.

Running stages are non-preemptive. A cleaner can switch from floor work to washing a pitcher only at a declared micro-task/checkpoint boundary. Cancellation of an API call does not prove that an external effect or provider charge did not occur.

17.2 Authoritative Admission Contract

In one transaction, admission verifies current readiness, compatible active workers, station capacity, resource state and ownership, spending/rate allowance, batch membership, idempotency, and current claim generations. It either commits a documented feasible subset or rejects the proposal; the chosen behavior is fixed for every benchmark policy. Stale proposals trigger revalidation and replanning.

Proposition 2 (Conditional admission safety)**.** *Suppose the initial authoritative state is feasible, admission and resource release are atomic, only admitted tickets execute, and uncertain effects retain their claims until safe reconciliation. Then each committed transition preserves task-claim exclusivity, readiness at dispatch, and the modeled admitted-capacity bounds.*

Proof. An admission transition checks the invariants against current state before creating claims. A completion or release transition updates only claims owned by the current fenced attempt and follows the resource lifecycle. Idempotent replay does not create an additional transition. Induction over committed transitions preserves the stated invariants. □

The proposition concerns admitted state. Preventing an expired worker from producing an external effect additionally requires destination-side fencing, deduplication, or reconciliation. Forecast accuracy is not a precondition for authoritative admission safety.

18 Elastic Staffing and Bottleneck Response

18.1 Worker Lifecycle

Each potential worker is in one of

$$\{\text{Off}, \text{Starting}, \text{ActiveIdle}, \text{ActiveBusy}, \text{Draining}\}.$$

Workers with ongoing work remain counted while draining but receive no new assignments. Let $n_k(t)$ count active or draining workers with home role k. Normal operation maintains

$$1 \le n_k(t) \le K_k, \qquad n_k(0) = 1, \qquad 4 \le \sum_k n_k(t) \le 7. \tag{68}$$

Starting workers become assignable only after their recorded startup delay. A Starting worker still occupies its unique potential position and provisioning allowance, although it is excluded from active-worker counts. A position cannot be activated twice or overlap a draining incumbent. Failure and replacement are explicit exception/recovery transitions.

For a feasible activation action v, estimate

$$\Delta_{\text{on}}(v) = \widehat{Q}_t(\text{activate } v) - \widehat{Q}_t(\text{current fleet}). \tag{69}$$

The forecast includes startup latency, setup/context cost, expected extra requests, station limits, and the same target weights. Activate only when $\Delta_{\text{on}}(v) > \theta_{\text{on}}$ and all provisioning limits permit it.

Using the same horizon, target, and rollout, define retention benefit

$$\Delta_{\text{keep}}(v) = \widehat{Q}_t(\text{keep } v) - \widehat{Q}_t(\text{drain } v). \tag{70}$$

Scale down an optional worker when $\Delta_{\text{keep}}(v) < \theta_{\text{off}}$ for a minimum idle interval. Use $0 < \theta_{\text{off}} < \theta_{\text{on}}$ plus a cooldown to reduce oscillation. Thus an idle optional worker with zero retention benefit can drain even when idle token charges are zero. Transition through Draining and then Off after its admitted work and resource obligations finish. Keep one home-role worker per role.

The joint decision is (staffing action, assignment set). The local packing derivation conditions on one current/forecast fleet. For the demo, at most three optional positions give eight potential on/off fleet configurations before lifecycle restrictions; bounded outer enumeration can compare them. Activation is not free instantaneous capacity inside the current packing problem.

An idle logical agent does not automatically incur API token charges. Only actual requests and explicitly measured hosting/setup costs are charged. Consequently, scaling down saves API money only when it avoids real calls, repeated context, or other measured spending.

18.2 Role-Specific Examples

Role	Observed/forecast bottleneck	Eligible scheduling response
Cashier	Preparation backlog threatens cart deadlines	Defer new intake stages or assign a permitted hand-off task; retain original arrival clocks
Barista	Complex recipes require scarce clean equipment	Prioritize prerequisite preparation/cleaning; consider an optional barista only if equipment and station capacity permit useful parallelism
Server	No near-term ready deliveries	Assign compatible cup preparation or cashier assistance if permissions and duty-station capacity allow
Cleaner	Clean pitchers/cups approaching depletion	Finish the current safe micro-task, then restore the equipment with greatest downstream benefit

These are consequences to test, not unconditional hard-coded role rewards. All roles share the terminal objective; independent local rewards must not incentivize taking orders or creating intermediate work without delivery.

19 Event-Driven Algorithm and Computation Budget

Inputs: horizon H, inner iteration cap M, planning-time cap $\tau_{\max}$, objective weights, resource scales, rollout policy, activation thresholds, and bounded retry rules.

1. **Observe and reconcile.** Apply arrivals, committed outcomes, resource releases, pricing/usage records, and worker lifecycle events. Retain uncertain claims and unresolved usage reservations.
2. **Snapshot.** Read authoritative state X_t^{ν} and expand newly revealed recipe stages using unique task identities.

3. **Forecast.** Simulate the known DAG and legal staffing choices. Estimate downstream value, deadline exposure, execution costs, and coordination overhead under Equation (20).
4. **Build the packing instance.** Enumerate current ready task–worker options, incremental demands, and residual capacities.
5. **Update prices.** Warm-start compatible dual dimensions; repeat the independent task oracle and Equation (67) for at most M iterations or until the planning-time cap is reached. Keep the best evaluated candidates.
6. **Recover a feasible action.** Build and jointly evaluate integer candidate sets, including waiting and any required recovery action. Choose a legal staffing transition if beneficial.
7. **Admit atomically.** Revalidate the current state and commit claims, budget/rate reservations, fencing generations, and dispatch tickets. Discard rejected or stale portions according to the contract.
8. **Execute.** Role agents consume admitted tickets, call bounded tools, and submit fenced outcomes. Record every API attempt.
9. **Replan on events.** Trigger on arrivals, outcomes, failures, deadline-risk changes, resource recovery, and startup completion. Coalesce bursts. Use a bounded timer for pending releases and retries; no API request is required merely to advance a display tick.

If $N = |\mathcal{E}_t|$, $D = |\mathcal{D}_t^{\text{pack}}|$, and each option touches at most $d \leq D$ dimensions, M dual-oracle iterations cost $O(MNd)$ plus $O(MD)$ price updates. A sorting-based recovery costs $O(N \log N + Nd)$ before rollout evaluations. If L rollouts each cost C_{roll}, forecasting adds $O(LC_{\text{roll}})$. These are implementation operation counts, not empirical latency guarantees.

Use bounded iterations, candidate pruning, cached DAG summaries, and event coalescing to control coordination overhead. A time-cap fallback selects a precomputed feasible FCFS/earliest-deadline action and still passes admission. Log fallback frequency. The cost of failed or abandoned planning is included in coordination metrics.

20 Anthropic Agent Execution and Tokenomics

The four role types use bounded Anthropic tool-use loops. Prompts expose the role, relevant observed state, permitted capabilities, admitted ticket, and a limited tool set. The model proposes semantic actions; authoritative tools validate them. Model text claiming completion does not commit a task success. A role agent can invoke only the operations allowed by the current ticket, ownership generation, and permissions.

SpacetimeDB stores authoritative business state; the external TypeScript service owns Anthropic requests. Reducers provide transactional state changes and do not perform network requests [5]. UI subscriptions display committed views of carts, tasks, resources, workers, decisions, and costs.

20.1 Per-Attempt Usage Ledger

For every physical request attempt j, persist run, role, task/batch, operation and request identifiers, model identifier, timestamps, outcome, pricing version, usage source, and accounting status. Logical retries and physical SDK retries are distinguished. Choose one retry owner so hidden retries do not disappear from instrumentation.

Using non-overlapping usage categories, define

$$c_j^{\text{API}} = 10^{-6}(p_j^{\text{in}} n_j^{\text{in}} + p_j^{\text{out}} n_j^{\text{out}} + p_j^{\text{read}} n_j^{\text{read}} + \sum_{\tau \in \mathcal{T}_j} p_{j,\tau}^{\text{write}} n_{j,\tau}^{\text{write}}) + c_j^{\text{provider tools}}. \tag{71}$$

Rates are dollars per million tokens for the recorded model, provider, usage tier, and applicable cache lifetime. Cache reads, cache creation, ordinary input, and output are priced according to the current provider contract [3, 4]. If cache-write totals and their lifetime-specific components are both present, use the components or the aggregate, never both. Prices are configuration data, not fixed constants in the mathematical model.

Known usage from failed, invalid, or superseded attempts is still charged. Missing usage remains estimated or unknown and is reconciled; a timeout is not evidence of zero spend. The ledger reports usage-derived spending separately from invoice reconciliation and any unresolved amount. Persist telemetry with idempotent keys and a durable outbox when a write temporarily fails.

For a shared batch call, choose attribution weights

$$\alpha_{jc} \geq 0, \qquad \sum_{c \in \text{AffectedCarts}(j)} \alpha_{jc} = 1. \tag{72}$$

Then $c_{jc} = \alpha_{jc} c_j^{\text{API}}$ is cart attribution, while run cost counts c_j^{API} exactly once. Attribution does not duplicate charges. Planner calls belong to coordination cost; role-execution calls belong to execution cost. Simulated task duration and real API wall time remain separate recorded quantities.

21 Claude Comparison and Minimal Benchmark Protocol

The comparison asks whether the scheduling controller changes workflow outcomes under the same environment. It is not an estimate of general model intelligence. Anthropic can power role execution under both policies while a separate Claude planner supplies the comparison schedule.

21.1 Controllers and Shared Conditions

Compare FCFS, a Claude scheduling controller, and the proposed SYNC controller. For each paired workload, hold constant:

- external arrivals, cart contents, recipes, initial resource states, deadlines, horizon, and target weights;
- worker capabilities, station limits, legal activation actions, startup assumptions, and permitted tools;
- admission/fencing rules, failure scenario, and execution semantics;
- visible state and future-information restrictions.

Claude receives a structured current-state snapshot and returns proposed assignments/staffing actions through a schema. It passes the same admission gate. Record model, prompt version, context limits, sampling settings, cache configuration, planning frequency, invalid proposals, and retry policy. Low temperature is not a guarantee of deterministic behavior.

Include Claude planning tokens and its wall time. Include SYNC's dual updates, forecasts, database coordination, and any planner requests. Shared role-agent costs belong to the policy run in which they occurred. If the controllers induce different contexts, tool calls, or execution paths, actual role-agent outcomes may differ; record those differences.

21.2 Workloads and Ablations

Use paired workloads covering steady arrivals, bursts, repeated quantities, mixed complex recipes, near-deadline carts, scarce clean pitchers, tray depletion, API failures, and demand falling after a burst. Include cleanup and reconciliation at shutdown. If cleanup continues after score horizon T, report that post-horizon cost and completion separately rather than moving it invisibly outside the measurement boundary.

Use a declared two-part protocol when appropriate:

1. **Controlled replay:** deterministic exogenous durations and failures isolate scheduling behavior; report simulation metrics.
2. **Live API runs:** actual Anthropic agents establish observed token usage, failure behavior, and wall-clock performance.

Replay costs are estimates unless backed by actual calls. A live API benchmark does not become deterministic merely because arrival seeds match. Repeat live runs and report variability.

To separate mechanisms, compare SYNC with fixed four workers, with elastic staffing, without scarcity prices, and without DAG lookahead. An elastic-versus-elastic comparison isolates scheduling more clearly than comparing seven available SYNC workers against a fixed-four baseline.

21.3 Measurements and Reporting

Measurement	Definition/interpretation
Primary objective	J_T with the same declared weights for every controller
Completion	Delivered carts, terminal value, and unfinished carts at T
SLA	Late deliveries and overdue unfinished carts; arrival clocks unchanged
Latency	Completed-cart median/p95 plus unfinished count and censored waiting
API spending	Usage-derived dollars, estimates/unknowns separately, all attempts
Planning	Per-event median/p95 and total wall time, including forecast and price updates
Coordination	Measured planner API/compute cost and decision/admission overhead
Resources	Waiting, utilization, clean/dirty inventory, starvation indicators
Elasticity	Active-worker trajectory, startups, drains, and avoidable oscillations
Correctness	Dependency/ownership violations, rejected stale results, duplicate effects

Report paired per-workload differences and variability across seeds or repeated live trials. Relative objective improvement can be misleading when the baseline score is zero or negative; report absolute $J_T^{\mathrm{SYNC}} - J_T^{\mathrm{Claude}}$ and its components. Use percentage reductions only when the denominator and direction are meaningful. Conditioning latency only on delivered carts can favor a controller that leaves difficult carts unfinished, so always show unfinished outcomes.

22 Limitations, Assumptions, and Implementation Status

The finite workload, resource inventory, and worker pool make experiments bounded, not automatically easy to solve. Current assumptions include non-preemptive stages, bounded

Metric	FCFS	Claude	SYNC
Terminal value	32	32	32
Objective J_T	12.670	17.882	17.613
Delivered / unfinished	32 / 0	32 / 0	32 / 0
Deadline violations	0	0	0
API cost (USD, recorded)	0	0	0
Reported coordinator time (ms)	0	15864.060	0
Invariant violations	—	—	—

Table 1: FCFS, Claude, and SYNC comparison. Dashes denote unavailable measurements, not zeros. The companion exporter can replace this block from measured SpacetimeDB results without changing Equation (20).

agent/tool loops, explicit resource ownership, and a declared cost model. Uncertain duration and token usage require forecasting and reconciliation.

The packing LP is a tractable relaxation of a local additive surrogate. Integer recovery, delayed terminal rewards, batching, stateful cleaning, forecast error, and dynamic arrivals separate that surrogate from the complete online problem. There is no general competitive-ratio, global-optimality, or unconditional progress claim in this specification.

Safety follows from the admission contract under its stated assumptions; performance must be measured. A minimal Claude comparison establishes behavior only on the tested settings. Claims about lower spend require actual comparable spending data, including coordination.

This document specifies the proposed policy in sufficient detail to implement it. It does not certify that every policy component is already present in a repository. The final implementation report should record the source commit, completed components, parameter configuration, benchmark traces, and unresolved limitations. Numerical results are filled only from actual experiment logs.

23 Explanation

SYNC asks: *which action helps the whole team finish the most valuable work, after paying for lateness, execution, and coordination?*

The coffee shop makes the dependencies visible. The cashier can create a barista backlog; the barista can be blocked by a dirty pitcher; the server can be idle while no drinks are ready. Local busyness does not itself produce a delivered cart.

Online control responds to changing work. Multidimensional constraints require every selected task to fit all the resources it needs. Dual prices make scarce capacity more expensive in the scheduling decision. Full-DAG forecasting estimates how a preparation or cleaning step contributes to eventual delivery. Atomic admission protects the execution rules.

Anthropic supplies the role agents and a Claude scheduling comparator. The experiment measures whether explicit coordination improves the shared objective enough to justify its own tokens and planning time. The same structure can apply to other agent workflows that share tools, execution slots, external-service quotas, or budgets.

A Minimum Reproducibility Record

For each run retain the controller and source version; workload/arrival trace; recipe and task schema versions; physical resource inventory; worker capability matrix; horizon and clock semantics; objective weights; price scales and initialization; M, H, and time caps; rollout settings; activation thresholds and delays; model/prompt/pricing versions; token, retry, tool, and cache limits; all admitted/rejected proposals; fenced outcomes; usage completeness; and raw per-run metrics.

Initial prices may be zero. Fix J_{ref} and resource scales from declared physical units, tune hyperparameters on separate development workloads, and freeze them before evaluating held-out traces. Demonstrate that five identical requested items remain five unique instances, that no cart delivers before all item outputs exist, that uncertain work retains claims, and that optional workers drain back toward the baseline fleet after demand subsides.

References

[1] N. Buchbinder and J. Naor. *Online Primal-Dual Algorithms for Covering and Packing*. Mathematics of Operations Research, 34(2):270–286, 2009. https://doi.org/10.1287/moor.1080.0363.

[2] B. Sun, L. Yang, M. Hajiesmaili, A. Wierman, J. C. S. Lui, D. Towsley, and D. H. K. Tsang. *The Online Knapsack Problem with Departures*. arXiv:2209.11934, version 3, 2023. https://arxiv.org/abs/2209.11934.

[3] Anthropic. *Prompt caching*. Claude Platform documentation. https://platform.claude.com/docs/en/build-with-claude/prompt-caching.

[4] Anthropic. *Pricing*. Claude Platform documentation. https://platform.claude.com/docs/en/about-claude/pricing.

[5] Clockwork Labs. *Reducers: Overview*. SpacetimeDB documentation. https://spacetimedb.com/docs/functions/reducers/.

[6] *Performance-effective and low-complexity task scheduling for heterogeneous computing* https://ieeexplore.ieee.org/document/993206